

Algorithmique et Programmation orientée-objet

Ch.2 – Classes et Objets

Bruno Quoitin
(bruno.quoitin@umons.ac.be)

Chapitre 2

- **Objectifs du Chapitre**

- Principes de **programmation orientée-objet** (P.O.O.)
 - classe, abstraction, encapsulation, héritage, polymorphisme
- P.O.O. en Java
 - Définition de classes
 - Utilisation de classes
 - Différence entre valeur et référence
 - Attributs et méthodes de classe (static)

Table des Matières

- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**

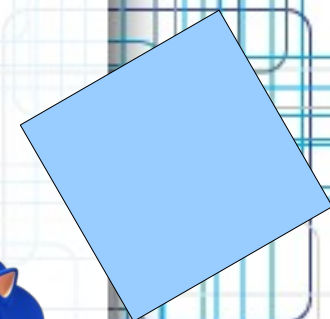
Introduction

- **Paradigme de programmation**
 - Méthodologie et théorie définissant la manière de penser et de structurer un programme informatique. Plusieurs paradigmes sont très répandus
 - **Programmation impérative** ou **procédurale** (1^{er} cours BAB1)
 - fonctions/procédures traitent et modifient les données
 - programmes = typiquement séquences d'instructions
 - **Programmation fonctionnelle** (cf. cours de BAB3)
 - fonctions sont des éléments de 1^{ère} classe du langage
 - données immuables; création de nouvelles données par application de fonctions
 - **Programmation logique** (cf. cours de BAB3)
 - définition de faits et de règles logiques, utilisées pour dériver de nouveaux résultats
 - ...

Introduction

- **Programmation Orientée-Objet**

- La programmation orientée-objet vise à modéliser un problème par un ensemble d'objets ayant chacun un comportement bien défini et qui interagissent entre eux.
- Un **objet** *représente* une entité réelle ou abstraite.
 - étudiant, cours, horaire, bulletin de notes, ...
 - forme géométrique dans un espace à 2 dimensions
 - compte bancaire, transaction
 - personnage d'un jeu vidéo
 - profil d'un utilisateur sur un réseau social
 - annuaire téléphonique, chaque entrée d'un annuaire
 - ...



Abstraction

prénom: Jeremy
nom: D*****
âge: ***
cheveux: noirs
yeux: bruns
pointure: 43
...

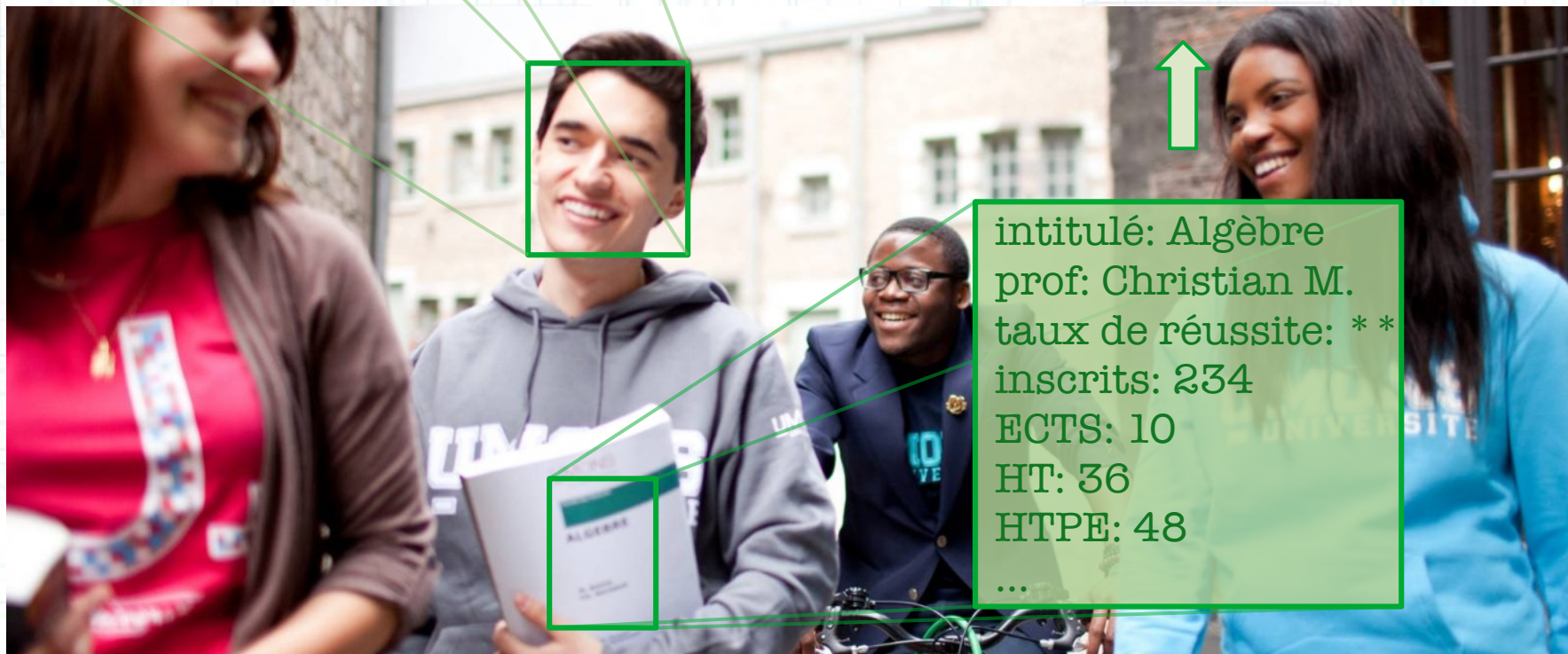


Etudiant 1	
nom	D*****
prenom	Jeremy
cours []	Cours 1, ...
notes []	**, **, ...

Cours 1	
intitulé	Algèbre
crédits	10



intitulé: Algèbre
prof: Christian M.
taux de réussite: **
inscrits: 234
ECTS: 10
HT: 36
HTPE: 48
...



1400 nouveaux étudiants à l'UMONS (2022)



...

Erreur

prénom: Arnaud
nom: B*****
âge: ***
cheveux: blonds
yeux: gris
pointure: 42

prénom: Renée
nom: R*****
âge: ***
cheveux: blonds
yeux: verts
pointure: 41

prénom: Liesbeth
nom: S*****
âge: ***
cheveux: noirs
yeux: bruns
pointure: 38

prénom: Galadriel
nom: A*****
âge: ***
cheveux: blonds
yeux: bleux
pointure: 40

Classe

prénom: Jeremy
nom: D*****
âge: ***
cheveux: noirs
yeux: bruns
pointure: 43
...

prénom: Galadriel
nom: A*****
âge: ***
cheveux: blonds
yeux: bleus
pointure: 40
...

prénom: Liesbeth
nom: S*****
âge: ***
cheveux: noirs
yeux: bruns
pointure: 38
...

prénom: Renée
nom: R*****
âge: ***
cheveux: blonds
yeux: verts
pointure: 41
...

prénom: Arnaud
nom: B*****
âge: ***
cheveux: blonds
yeux: gris
pointure: 42
...

1395 autres
étudiants ...

Etudiant 1	
nom	D*****
prenom	Jeremy
cours []	Cours 1, ...
notes []	**, **, ...

Etudiant 2	
nom	A*****
prenom	Galadriel
cours []	Cours 1, ...
notes []	**, **, ...

Etudiant 3	
nom	S*****
prenom	Liesbeth
cours []	Cours 1, ...
notes []	**, **, ...

Etudiant 4	
nom	R*****
prenom	Renée
cours []	Cours 1, ...
notes []	**, **, ...

Etudiant 5	
nom	B*****
prenom	Arnaud
cours []	Cours 1, ...
notes []	**, **, ...

Structure commune
à tout étudiant

Etudiant	
nom: String	
prenom: String	
cours: Cours []	
notes: double []	
double getMoyenne()	
boolean aReussi()	

Introduction

- **Principes de la P.O.O.**
 - Bien que le concept **d'objet** soit au coeur de la P.O.O., d'autres principes fondamentaux y sont intimement liés.

ce chapitre

- **Abstraction** : modélisation (avec des objets) en ne conservant que ce qui est nécessaire au problème. Séparation entre l'implémentation et son contrat (interface).
- **Classe** : description de l'organisation et du comportement partagés par un ensemble d'objets.
- **Encapsulation** : limitation de l'accès aux données internes à une classe, en distinguant ce qui est public ou pas.

Chapitre 4

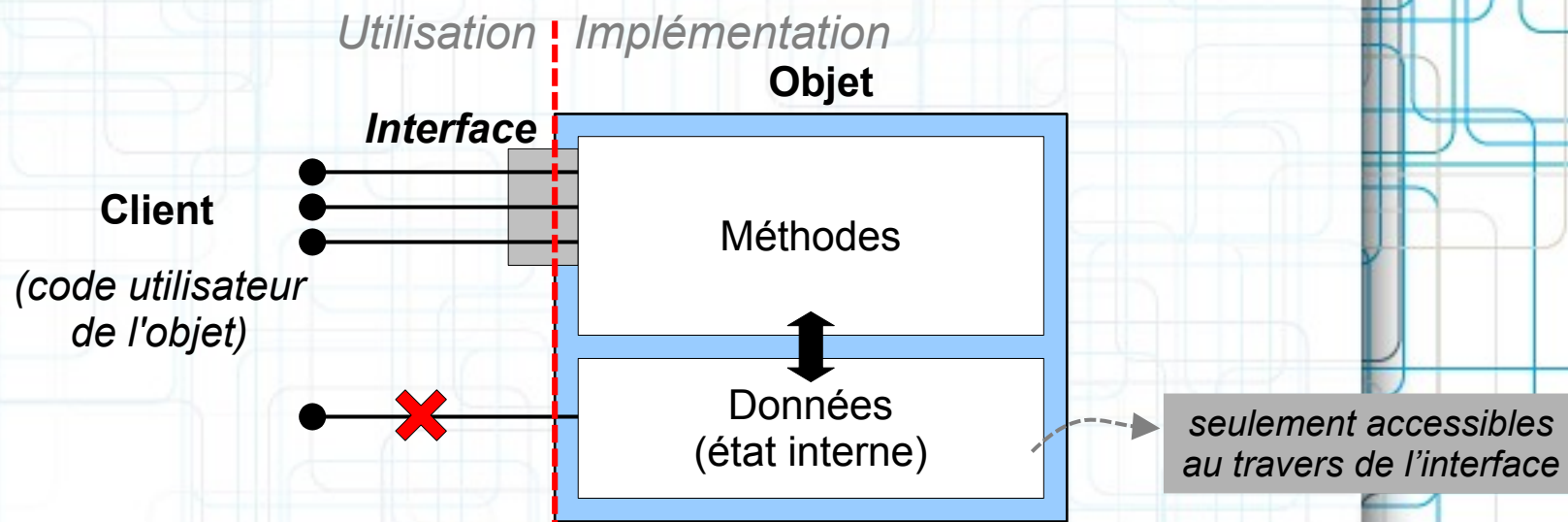
- **Héritage** : définition d'une classe sur base de celle d'une autre classe.
- **Polymorphisme** : différentes implémentations possibles tout en partageant une même interface (ou un même type).

Note : les étudiants curieux sont invités à lire l'article *The Quarks of Object Oriented Development* publié par Deborah J. Armstrong dans Communications of the ACM, en Février 2006

Introduction

- **Encapsulation**

- L'encapsulation est l'un des principes fondamentaux de la P.O.O. Il vise à **séparer** autant que possible **utilisation et implémentation** d'un objet.
 - Modularité : les objets sont indépendants les uns des autres
 - *Separation of concerns* : possible de changer l'intérieur d'un objet sans affecter son utilisation
 - Simplicité : utilisation d'un objet sans devoir connaître ses détails internes (boîte noire)



Introduction

- Héritage

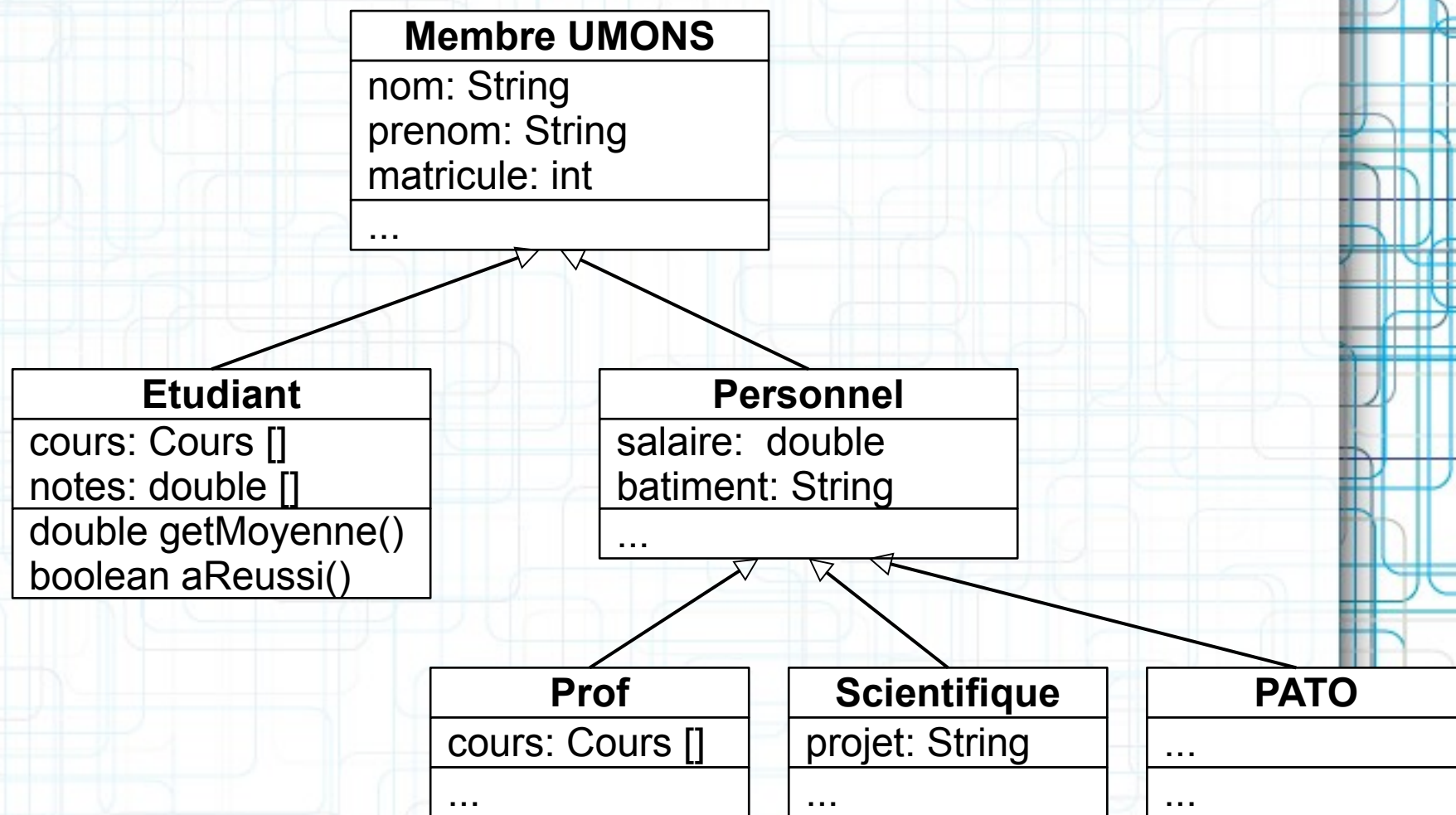


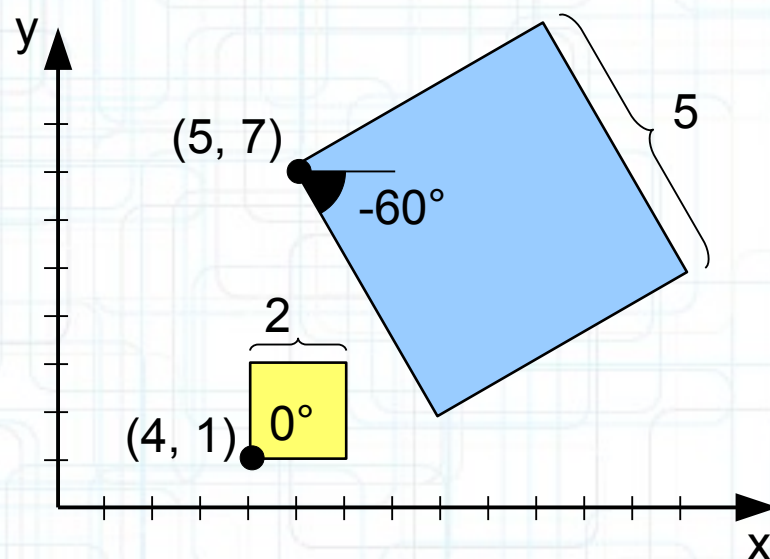
Table des Matières

- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**

Exemple introductif

- **Carrés en 2D**

- Comment modéliser des carrés dans un espace à 2 dimensions ?



Modélisation du carré bleu

- abscisse= 5
- ordonnée= 7
- angle= -60°
- longueur du côté= 5

Modélisation du carré jaune

- abscisse= 4
- ordonnée= 1
- angle= 0°
- longueur du côté= 2

- Commençons avec une approche impérative...

Exemple introductif

- Carrés en 2D

- Dans une approche impérative, on pourrait modéliser l'état des carrés avec un ensemble de variables, comme ceci :

double carreBleuX;	5
double carreBleuY;	7
double carreBleuAngle;	-60
double carreBleuCote;	5

double carreJauneX;	4
double carreJauneY;	1
double carreJauneAngle;	0
double carreJauneCote;	2

- Tandis que leur **comportement** pourrait être modélisé avec des fonctions, comme ceci :

```
void carreBleuDeplacer(double dX, double dY) {  
    carreBleuX += dX;  
    carreBleuY += dY;  
}  
  
void carreBleuTourner(double dAngle) {  
    carreBleuAngle = (carreBleuAngle + dAngle) % 360;  
}
```

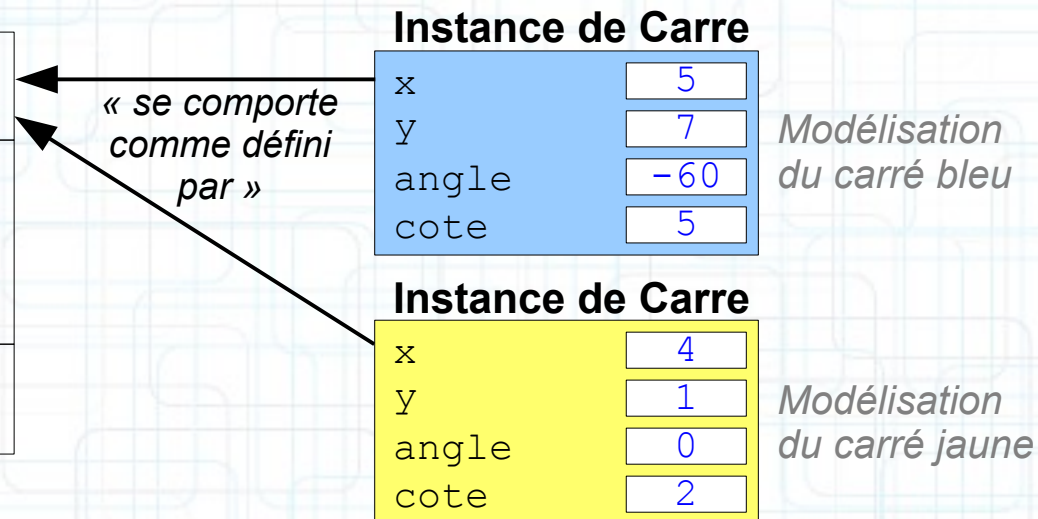

Exemple introductif

- **Carrés en 2D**

- Cette solution est **frustrante**.
 - Les carrés ont la même structure, pourtant on doit la redéfinir pour chaque nouveau carré.
 - Les fonctions ne travaillent que sur un carré, nécessité de répéter le code (copier-coller) + adapter les variables.
- Avec une **approche POO**, la *structure* des carrés et leur *comportement* sont définis 1 seule fois, dans une classe.

Classe Carré
Modélisation d'un carré quelconque
<pre>double x; double y; double angle; double cote;</pre>
<pre>void déplacer(carre, dx, dy); void tourner(carre, dAngle);</pre>

sur quelle instance ?



Introduction

- **Terminologie**

- **Classe** : modèle d'un ensemble d'entités ayant la même structure et la même implémentation.
- **Objet** : modèle d'une entité particulière / individuelle.
- **Instance** d'une classe C : objet construit à partir de C.
- **Attributs** : variables d'une classe ou d'un objet
- **Méthodes** : fonctions propres à une classe ou à un objet.
- **Membres** : attributs et méthodes d'une classe ou d'un objet.

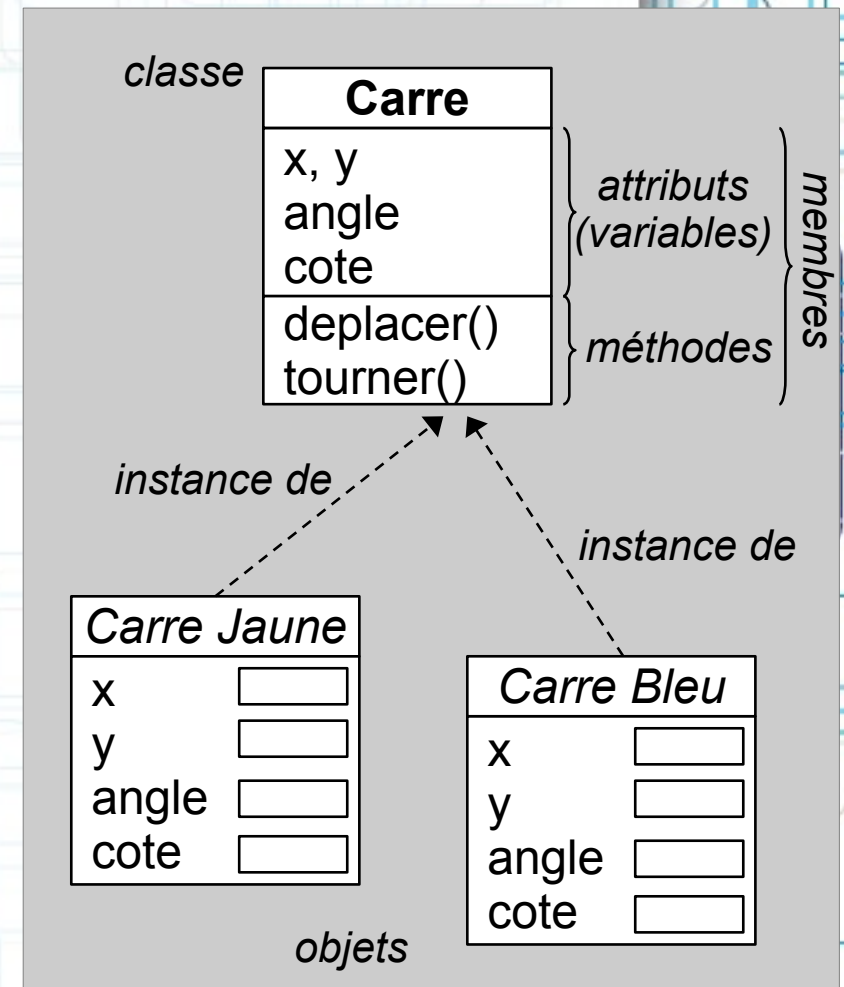


Table des Matières

- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**

Classe

- **Définition d'une classe**

- Une classe est une structure de données qui regroupe des attributs (des variables) et des méthodes.

Carre.java

```
public class Carre {  
    public double x, y;  
    public double angle;  
    public double longueurCote;  
  
    public void deplacer(double dX, double dY) {  
        x += dX;  
        y += dY;  
    }  
  
    public void tourner(double dAngle) {  
        angle = (angle + dAngle) % 360;  
    }  
}
```

Spécificateurs
d'accès.

Attributs
(variables d'instance)

Méthodes
(méthodes d'instance)

x, y et angle ne sont pas des variables locales, mais des **attributs de la classe**. Leur portée s'étend à l'ensemble de la classe.

Au contraire des méthodes présentées au Chapitre 1, ces méthodes **ne sont pas** définies avec le modificateur **static**.

Classe

- **Conventions**

- **1 classe = 1 fichier**
- Nom de classe = identifiant. Par convention,
 - le nom **commence par une lettre majuscule**.
 - le nom suit la **forme « chameau »** lorsqu'il est composé de plusieurs mots.
- Exemples
 - Carre, Etudiant, Compte, ArrayList, ...
- 1 classe qui contient une méthode `main` avec la signature adéquate est **exécutable**

```
public static void main(String[] args)
```

Classe

- **Spécificateurs d'accès**

- Les définitions d'attributs et de méthodes sont précédés d'un **spécificateur d'accès**.

```
public double x, y;  
public void deplacer(double dX, double dY);
```

- Ceux-ci permettent de restreindre les classes qui peuvent accéder à ces attributs / appeler ces méthodes.

- **private** : accessible seulement depuis la classe où il est défini
- **public** : accessible par toute autre classe
- **protected** : variante de **private** (utilisé avec l'héritage – cf. Chapitre 4)
- *package* (pas de spécificateur) : accessible par les classes du même package

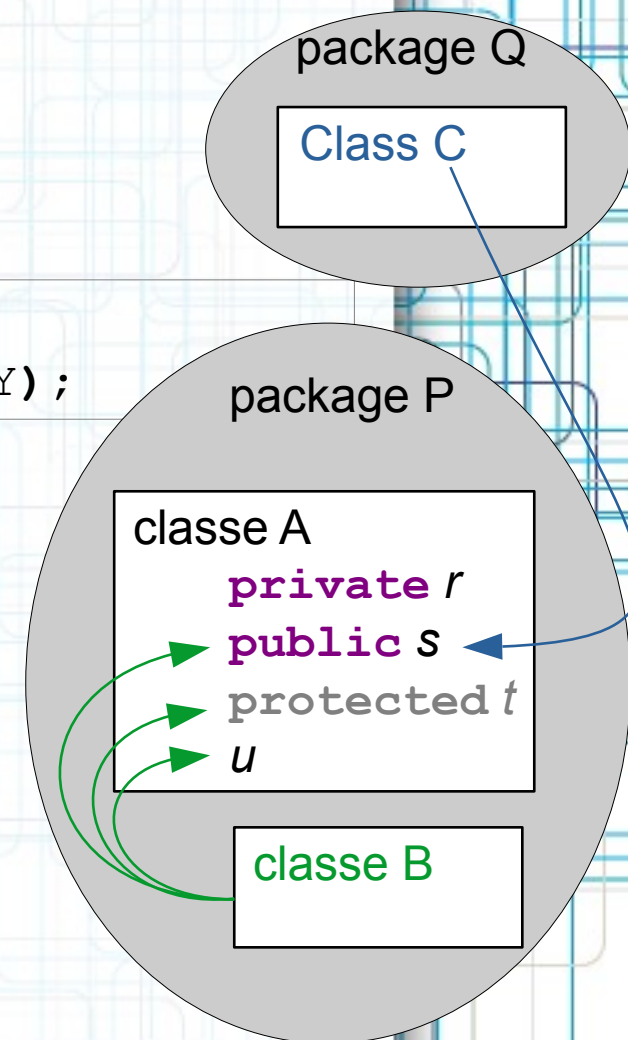


Table des Matières

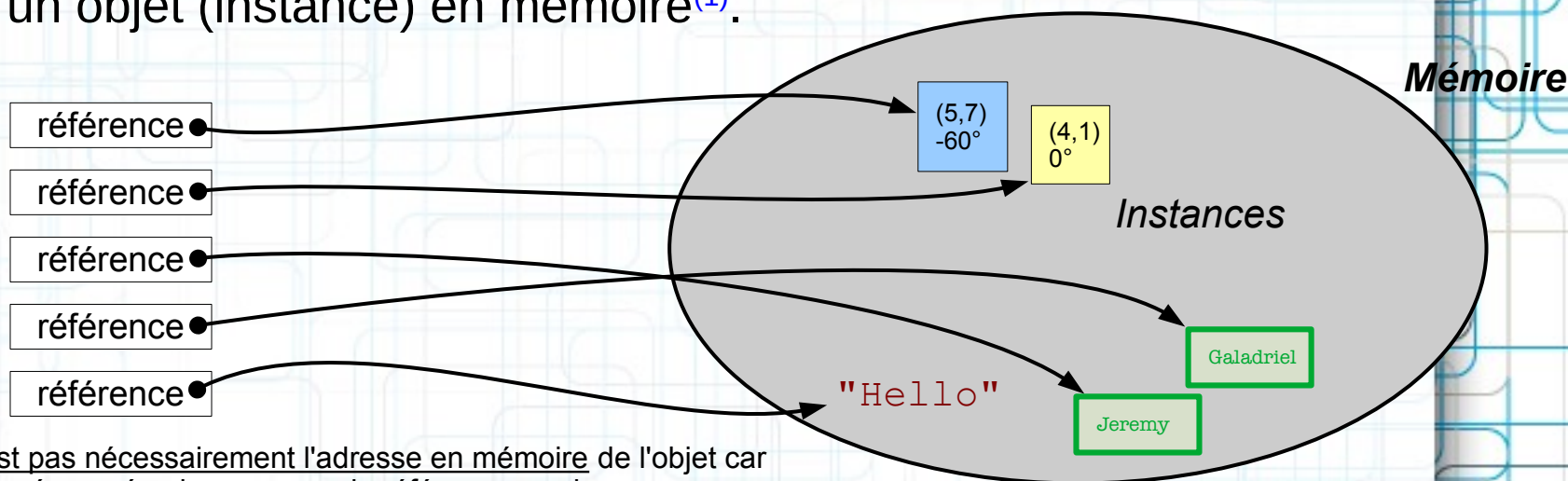
- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**



Instanciation

- **Création d'une instance**

- Une classe ne définit que la structure et le comportement d'un ensemble d'objets.
- Pour chaque objet, il faut **créer une instance** de la classe
 - 1). allouer l'espace mémoire nécessaire à l'instance ;
 - 2). initialiser l'instance via le **constructeur** de la classe ;
 - 3). obtenir une **référence** vers l'instance créée. Une référence est un identifiant utilisé par la machine virtuelle pour trouver un objet (instance) en mémoire⁽¹⁾.



(1) Une référence n'est pas nécessairement l'adresse en mémoire de l'objet car celui-ci peut être déplacé en mémoire sans que la référence ne change.

Instanciation

• Valeur versus référence

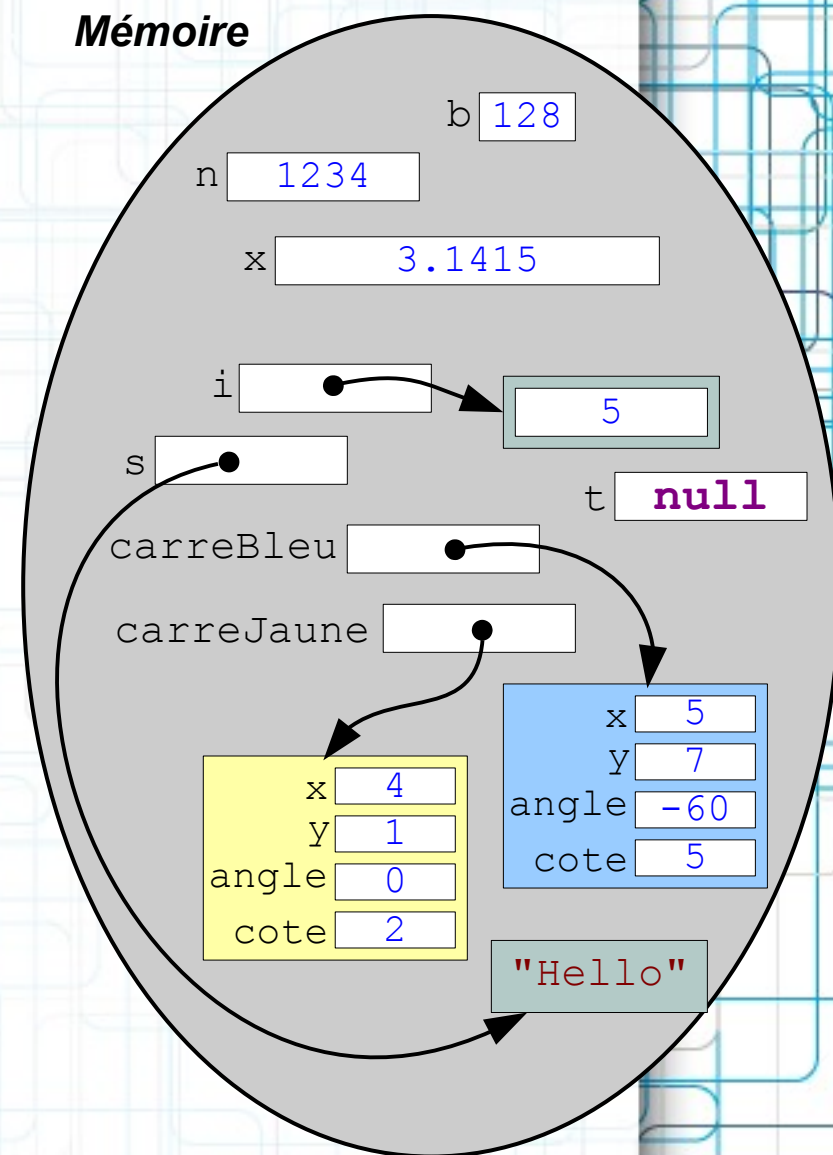
- Une variable de type primitif stocke une valeur

```
int n = 1234;
double x = 3.1415;
short b = 128;
```

- Une variable de type objet stocke une référence vers une valeur.

```
Carre carreJaune = ...;
Carre carreBleu = ...;
String s = "Hello";
Integer i = 5;
String t = null;
```

- Le littéral **null** désigne une référence invalide.

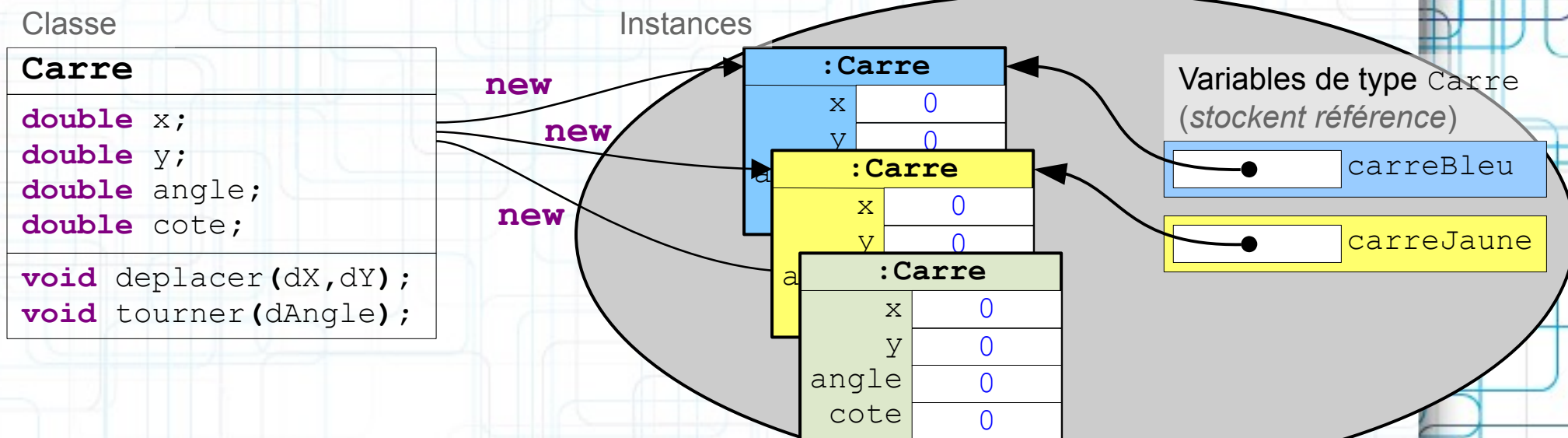


Instanciation

- **Création d'une instance**

- En pratique, créer une instance consiste à employer l'opérateur **new** avec le nom de la classe. Cet opérateur retourne une *référence* vers l'instance créée.
- Exemple : création de deux instances de la classe Carre.

```
Carre carreBleu = new Carre();
Carre carreJaune = new Carre();
...
```



Dé-allocation

- **Comment supprimer une instance ?**

- Il n'y a pas d'opérateur pour supprimer une instance. Les instances vivent-elles éternellement ?

```
Carre carreBleu = new Carre();  
...
```

- Une instance est supprimée **automatiquement** de la mémoire lorsqu'il n'existe plus aucune référence vers elle. Le mécanisme en charge de cette suppression est appelé « **ramasse-miettes** » (*garbage collector*). On ne peut pas contrôler quand celui-ci va libérer une instance de la mémoire.
- Pour supprimer une référence vers une instance
 - Affecter le littéral **null** à une variable de type référence

```
carreBleu = null;
```
 - Sortir du bloc où une variable locale de type référence est définie (la variable est perdue et donc la référence aussi)

Instanciation

- **Compatibilité des types références**

- A chaque fois qu'une valeur est affectée à une variable ou passée en paramètre d'une méthode, le compilateur vérifie que le type de la valeur est compatible avec celui de la variable ou de l'argument !
- A ce stade, nous considérons que **deux types objets sont compatibles s'ils sont les mêmes.**
- Exemple

```
Carre carreBleu = new Carre();  
Carre carre = carreBleu;  
Integer nombre = new Integer(5);  
Double d = 1.37;  
Carre autreCarre = "Carre";  
Carre carreJaune = new Livre();  
String msg = nombre;
```

Compatibles
(types identiques)

« Auto-boxing »
d'un primitif **double**
en objet **Double**

Incompatibles
(types différents)

- Cette règle sera adaptée lorsque nous verrons l'héritage et le polymorphisme au Chapitre 4.

Table des Matières

- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**

Constructeur

- **Constructeur**

- Chaque classe possède un constructeur.
 - Appelé lors de l'instanciation (**new**)
 - Responsable de l'initialisation de chaque instance
 - Ne retourne pas de valeur
 - A le même nom que la classe

```
public class Carre {
    /* ... */
    public Carre(double _x, double _y,
                 double _angle, double _cote) {
        x = _x;
        y = _y;
        angle = _angle;
        cote = _cote;
    }
}
```

identiques

```
Carre carreBleu = new Carre(5, 7, -60, 5);
Carre carreJaune = new Carre(4, 1, 0, 2);
```

:Carre	
x	5
y	7
angle	-60
cote	5

:Carre	
x	4
y	1
angle	0
cote	2

Constructeur

- **Constructeur par défaut**

- Si aucun constructeur n'est fourni, un **constructeur par défaut** est automatiquement créé.
- Celui-ci ne prend **pas de paramètre**.
- Il initialise toutes les variables d'instance de la classe avec des **valeurs par défaut**, dépendantes du type des variables.

Type d'attribut	Valeur par défaut
byte, short, int, long, float, double	0
char	'\u0000'
boolean	false
référence d'objet	null

Constructeur

- **Surcharge des constructeurs**

- Il est permis de définir **plusieurs constructeurs** dans une même classe (surcharge). Ces constructeurs doivent avoir des signatures différentes⁽¹⁾.
- Exemple

```
public Carre(double _x, double _y,  
             double _angle, double _cote) {  
    x = _x;  
    y = _y;  
    angle = _angle;  
    cote = _cote;  
}  
  
public Carre(double _x, double _y, double _cote) {  
    x = _x;  
    y = _y;  
    cote = _cote;  
    angle = 0;  
}
```

⁽¹⁾ de la même façon qu'avec la surcharge des méthodes (cf. Ch. 1).

Constructeur

- **Chaînage de constructeurs**

- Le mot-clé **this** permet d'invoquer un constructeur surchargé dans un autre constructeur de la même classe (chaînage de constructeurs).
- L'exemple précédent peut ainsi s'écrire

```
public Carre(double _x, double _y,  
             double _angle, double _cote) {  
    x = _x;  
    y = _y;  
    angle = _angle;  
    cote = _cote;  
}  
  
public Carre(double _x, double _y, double _cote) {  
    this(_x, _y, 0, _cote);  
}
```

- Contrainte : l'invocation d'un constructeur avec **this** DOIT avoir lieu à la première ligne d'un autre constructeur !

Table des Matières

- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**

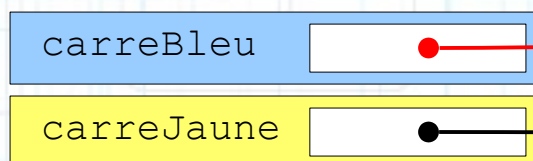
Accès aux attributs

- **Accès aux membres d'instance**

- A partir de la référence à une instance, il est bien évidemment possible d'accéder aux membres de celle-ci. Cela se fait à l'aide de l'**opérateur de dé-référencement** (« . »). Il s'agit d'un opérateur binaire qui prend d'une part la référence vers une instance et d'autre part le nom d'un attribut ou d'une méthode.

- **Accès à un attribut**

```
Carre carreBleu = new Carre(5, 7, -60, 5);
Carre carreJaune = new Carre(4, 1, 0, 2);
carreBleu.x = 19;
```



carreBleu . x

:Carre	
x	5 → 19
y	7
angle	-60
cote	5

:Carre	
x	4
y	1
angle	0
cote	2

L'opérateur de dé-référencement permet de désigner l'instance où se trouve l'attribut auquel on accède.

Dé-référencement

- **Vérifications lors du dé-référencement**

- **Membre existant** : la classe dont est tirée l'instance définit-elle le membre ?

Il s'agit d'une vérification statique effectuée par le compilateur.

```
Carre carreBleu = new Carre();
carreBleu.profondueur = 16;
```

???

```
UnknownSym.java:9: cannot find symbol
symbol : variable profondueur
location: class UnknownSym
        carreBleu.profondueur= 16;
```

Carre

```
double x,y
double angle
double longueurCote
void deplacer(double, double)
void tourner(double)
```

- **Référence valide** : la référence est-elle différente de **null** ?

Il s'agit d'une vérification dynamique effectuée par la JVM.

```
Carre carreVert = null;
carreVert.x = -9;
```

```
Exception in thread "main" java.lang.NullPointerException
    at InvalidRef.main(InvalidRef.java:11)
```

vous serez bientôt
familiers avec
cette erreur

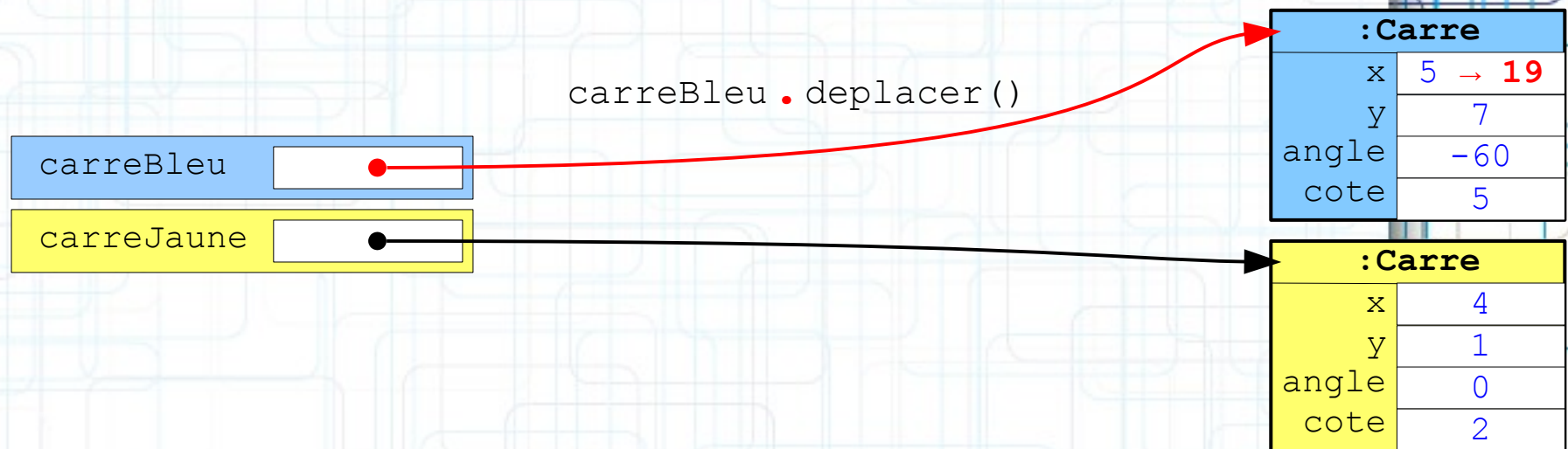


Appel de méthodes

- Appel d'une méthode

- Le même principe s'applique lors de l'appel d'une méthode. Pour le différencier d'un attribut, le nom de la méthode est suivi de parenthèses et des arguments de la méthode si elle en a.

```
Carre carreBleu = new Carre(5, 7, -60, 5);
Carre carreJaune = new Carre(4, 1, 0, 2);
carreBleu.deplacer(5, 7);
```



Appel de méthodes

- **Référence « this »**

- Lorsqu'une méthode est appelée, la référence de l'objet sur lequel elle s'applique lui est implicitement passée !!

```
public class Carre {  
    public double x, y;  
    public void deplacer(Carre inst,  
                        double dX, double dY) {  
        inst.x = inst.x + dX;  
        inst.y = inst.y + dY;  
    }  
}
```

```
Carre carreBleu= new Carre(5, 7, -60, 5);  
carreBleu.deplacer(carreBleu, 2, 2);
```



Attention, l'argument **inst** montré ici est juste une vue de l'esprit.

- Cette référence peut aussi être accédée explicitement, en utilisant le mot-clé **this**.

Référence « this »

- **Masquage d'attributs dans une méthode**

- Le **masquage d'attribut** (*shadowing*) est un phénomène qui se produit dans une méthode lorsque le même nom qu'un attribut est utilisé dans la déclaration d'**arguments** ou de **variables locales**

- Exemple

```
public class Carre {  
    private double x;  
    public void setX(double x)  
    {  
        x = x;  
    }  
}
```

Les deux « **x** » correspondent à l'argument **x** de la méthode.

Le paramètre **x** de la méthode **masque** l'attribut **x** à l'intérieur de la méthode.

- Solution

```
public class Carre {  
    private double x;  
    public void setX(double x)  
    {  
        this.x = x;  
    }  
}
```

Argument « **x** » de la méthode.

Attribut « **x** » de l'instance.

Référence « this »

- Masquage d'attributs dans un constructeur

```
public class Carre {  
    /* ... */  
    public Carre(double _x, double _y,  
                 double _angle, double _cote) {  
        x = _x;  
        y = _y;  
        angle = _angle;  
        cote = _cote;  
    }  
}
```

```
public class Carre {  
    /* ... */  
    public Carre(double x, double y,  
                 double angle, double cote) {  
        this.x = x;  
        this.y = y;  
        this.angle = angle;  
        this.cote = cote;  
    }  
}
```

Table des Matières

- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**

Références

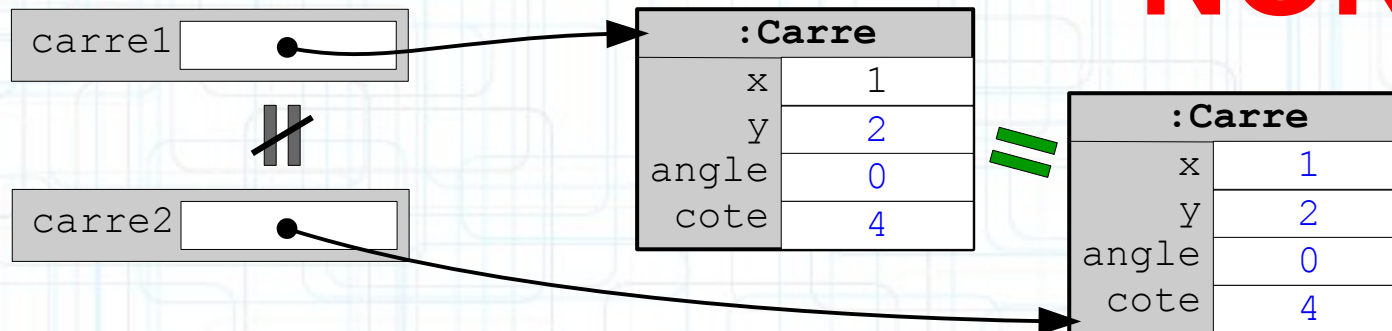
- **Comparaison d'objets**

- Une variable de type objet est une référence. Peut-on comparer ces deux instances comme suit ?

```
Carre carre1 = new Carre(1, 2, 0, 4);  
Carre carre2 = new Carre(1, 2, 0, 4);
```

```
carre1 == carre2
```

NON



L'opérateur `==` teste l'égalité des références.

`carre1` et `carre2` référencent deux instances différentes (mais de même contenu). Ces références sont donc différentes.

- Pour comparer la valeur des objets, il faut qu'ils adaptent la méthode `equals()` (à voir au Chapitre 4).

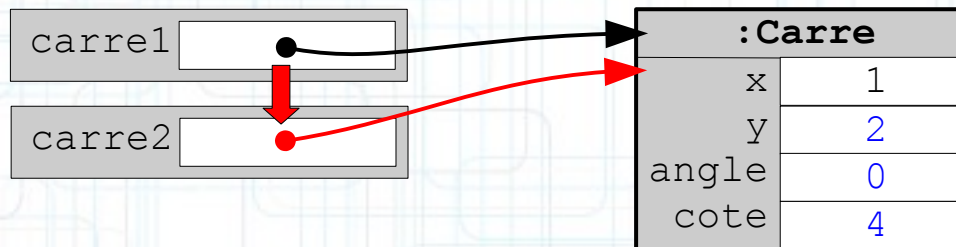
Références

- Copie d'un objet

- Peut-on créer une copie d'un objet comme suit ?

```
Carre carre1 = new Carre(1, 2, 0, 4);  
Carre carre2 = carre1;
```

NON



La référence vers le premier objet a été copiée dans la seconde variable. **Il n'y a pas eu création d'une copie de l'objet !**
La seconde référence est un **alias** de la première.

- Comment créer une copie ?

Références

- **Constructeur de copie**

- Pour créer une copie d'un objet, il faut en connaître la structure. C'est donc le concepteur de la classe qui doit fournir ce service.
- Une approche classique est de fournir un constructeur dédié pour la copie.

```
public class Carre {  
    public double x, y;  
    public double angle;  
    public double cote;  
    public Carre(double x, double y, double angle,  
                 double cote) {  
        this.x = x;  
        this.y = y;  
        this.angle = angle;  
        this.cote = cote;  
    }  
    public Carre(Carre c) {  
        this(c.x, c.y, c.angle, c.cote);  
    }  
}
```


Table des Matières

- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**

Modificateurs

- **Modificateurs**

- En plus d'un *spécificateur d'accès* (p.ex. **public**) la déclaration des membres d'une classe peut comporter des **modificateurs**.
- **final**
 - s'applique aux attributs
 - la valeur ne peut plus être modifiée une fois donnée
 - le constructeur DOIT donner une valeur initiale s'il n'y en a pas encore
- **static**
 - s'applique aux attributs et aux méthodes
 - associe le membre à la classe (pas aux instances)
 - une variable de classe existe en 1 seul exemplaire, partagé par les instances
 - une méthode de classe n'a pas de paramètre implicite **this**

Variable d'instance / de classe

```
public class Demo {
    public static int a = 0;
    public int b;
}
```

variable de classe

variable d'instance

```
Demo.a = 3;
Demo.b = 9;
Demo d1 = new Demo();
d1.b = 5;
System.out.println(d1.a);
Demo d2 = new Demo();
d2.b = 17;
```

	Variable d'instance	Variable de classe
Modificateur	—	static
Mémoire (allocation)	une par instance (à l'instanciation)	une seule (au chargement de la classe)
Accessibilité	<i>instance.v</i>	<i>classe.v</i> <i>instance.v</i>
Initialisation	par le constructeur	à la déclaration
Durée de vie	durée de l'instance	≈ durée du programme (dès chargement de la classe)

Variable d'instance / de classe

• Exemple

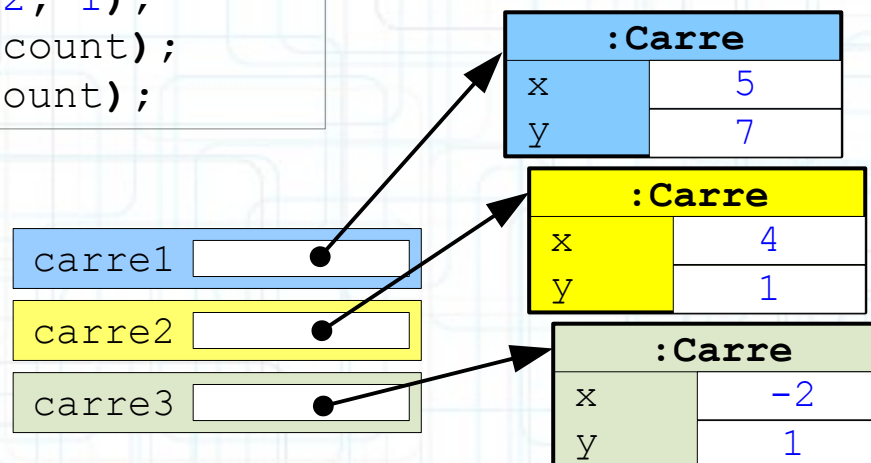
```
public class Carre {
    public static int count = 0;
    public double x, y;
    public Carre(double x, double y) {
        count++;
        this.x = x;
        this.y = y;
    }
}
```

```
Carre carrel = new Carre(5, 7);
Carre carre2 = new Carre(4, 1);
Carre carre3 = new Carre(-2, 1);
System.out.println(carrel.count);
System.out.println(Carre.count);
```

La variable `count` compte le nombre d'instance créées. Elle est incrémentée à chaque appel du constructeur.

Carre	
count: int	3
x, y: double	

variable de classe
exemplaire unique



variable d'instance
1 exemplaire / instance

Méthode d'instance / de classe

```
public class Demo {
    public static int a = 0;
    public int b;
    public static void a() {
        a = 3;
        this.b = 2;
    }
    public void b(this) {
        a = 17;
        this.b = 19;
    }
}
```

méthode de classe

méthode d'instance

Seules les méthodes d'instance reçoivent la référence **this** !

	Méthode d'instance	Méthode de classe
Modificateur	—	static
Accessibilité	<i>instance.m()</i>	<i>classe.m()</i> <i>instance.m()</i>
Mot clé this	oui	non
Accès membres d'instance	oui	non

Méthodes de classe

- **Utilité des méthodes de classe**

- Fonction pure : méthode dont toutes les entrées sont fournies en paramètres (ne dépend pas de l'état d'une instance).

```
Math.abs(x)  
Math.pow(a, b)  
Math.round(x)  
Math.sqrt(x)  
...
```

- Méthode accédant uniquement aux membres de classe

```
public class Carre {  
    private static int count= 0;  
    public Carre() {  
        count= count + 1;  
    }  
    public static int getInstanceCount () {  
        return count;  
    }  
}
```


Méthodes de classe

- **Utilité des méthodes de classe**

- Méthode qui doit pouvoir être appelée sans instance.
- Par exemple, la méthode `main` qui sert de point d'entrée du programme doit pouvoir être appelée avant toute création d'instance...

```
public static void main(String[] args) {  
    System.out.println("Je suis exécutable !");  
}
```

Jeu des erreurs

```
public class Carre {  
    public static int count= 0;  
    public double x, y;  
    public static double angle;  
  
    public Carre() {  
        count= count + 1;  
    }  
    public static void deplacer(double dX, double dY) {  
        x= x + dX;  
        y= y + dY;  
    }  
  
    public void tourner(double dAngle) {  
        angle= (angle + dAngle) % 360;  
    }  
}
```

méthode de
classe
n'a pas
accès aux
membres
d'instance !

angle devrait être un
membre d'instance

```
Carre c= new Carre();  
System.out.println(Carre.count);  
System.out.println(Carre.x);  
System.out.println(c.x);  
c.deplacer(5, 7);  
tourner(90);
```

x est un membre d'instance
⇒ nécessite la référence de
son instance !

tourner est un membre
d'instance ⇒ nécessite la
référence de son instance !

Table des Matières

- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**

Et en Python ?

- Exemple de classe

Pas de spécificateurs d'accès, mais des conventions

Attention danger, les membres de l'instance sont créés dynamiquement !

```
class Carre:
    count = 0
    def __init__(self, x, y, cote, angle=0):
        self.x = x
        self.y = y
        self.cote = cote
        self.angle = angle
        Carre.count += 1

    def deplacer(self, dX, dY):
        self.x += dX
        self.y += dY

    def tourner(self, dAngle):
        self.angle = (self.angle + dAngle) % 360

    @classmethod
    def get_instance_count(cls):
        return cls.count

carreBleu = Carre(5, 7, 5, -60)
carreJaune = Carre(4, 1, 2, 0)
```

variable de classe

constructeur

variables d'instance

méthodes d'instance

méthode de classe

instanciation

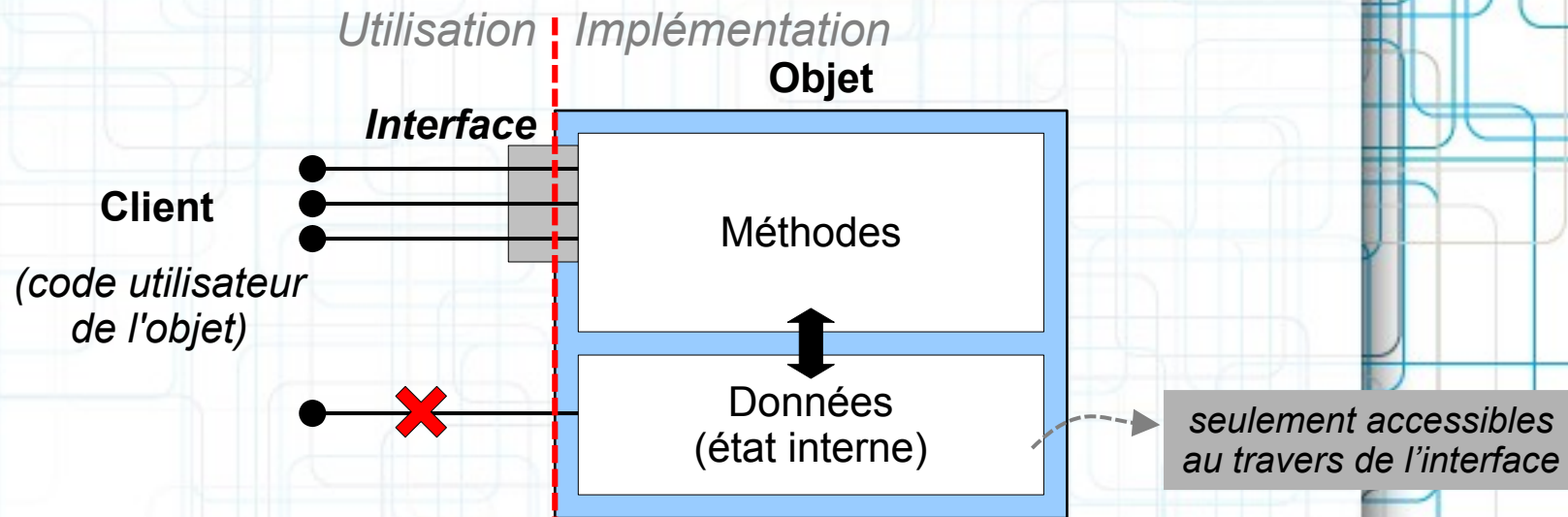
Table des Matières

- **Programmation orientée-objet**
 - Exemple introductif
- **Classe**
 - Instanciation
 - Constructeur
 - Accès aux variables et appels de méthodes
 - Comparaison, aliasing, copie,
 - Membres de classe
- **Et en Python ?**
- **Encapsulation**

Encapsulation

- **Principe**

- L'**encapsulation** est un des principes fondamentaux de la P.O.O. Il vise à regrouper des données et les méthodes qui y accèdent au sein d'un même objet.
- Ce principe veut aussi que l'objet soit une « *boîte noire* ».
- Les détails internes de l'objet ne doivent pas être connus de l'utilisateur de l'objet. Ce dernier ne devrait pouvoir accéder aux données de l'objet qu'au travers d'une interface bien définie.



Encapsulation

- **Comment procéder ?**
 - Certaines règles de bonne pratique existent.
 - Premièrement, on ne laisse pas les données directement accessibles à partir de l'extérieur de l'objet. Pour cela deux alternatives sont possibles :
 - rendre les **données privées**
 - garder les **données publiques**, mais **non modifiables** (utilisé dans les objets immuables tels que `String`, `Integer`, etc).
 - Si les données sont privées, on peut donner un accès contrôlé à celles-ci au travers de méthodes spécifiques appelées accesseurs (*getters*) et mutateurs (*setters*).

Encapsulation

- **Données privées et accesseurs**

- Dans cette première approche, le spécificateur d'accès **private** est utilisé pour rendre les variables et ou méthodes inaccessibles de l'extérieur de la classe.

```
public class Carre {  
    private double x, y;  
    public Carre(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
    public double getX() { return x; }  
    public double getY() { return y; }  
    public void deplacer(double dX, double dY) {  
        x += dX;  
        y += dY;  
    }  
}
```

```
Carre carre = new Carre(5, 7);  
carre.x = 3;  
carre.deplacer(3, 2);  
System.out.println(carre.getX(), carre.y);
```

Erreur, l'accès
aux membres
private est
empêché par le
compilateur.

Encapsulation

- **Classe immuable**

- Dans cette seconde approche, les attributs sont **public** mais **final**. Toute méthode modifiant l'objet doit en créer un nouveau (ici, `deplacer()`).

```
public class Carre {  
    public final double x, y;  
    public Carre(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
    public Carre deplacer(double dX, double dY) {  
        return new Carre(x + dX, y + dY);  
    }  
}
```

```
Carre carre = new Carre(4, 1);  
carre.x = -9;  
carre = carre.deplacer(-2, 3);  
System.out.println(carre.x, carre.y);
```

Erreur, l'écriture des membres **final** est empêchée par le compilateur.